

03_translation

Local + Cloud Professional Machine Translation for HelixLLM

Scope: Run professional-grade machine translation **LOCALLY** on a single **RTX 5090 (32 GB, Blackwell, sm_120)**, **Linux**, inside **rootless Podman** containers, to give the local LLM system (“HelixLLM”) strong translation capability — plus a unified API that also fronts the major cloud translation providers.

Author: Deep-research subagent (T1) · **Date:** 2026-07-06 · **Constitution:** §11.4.99 latest-source cross-reference, §11.4.161 rootless-container, §11.4.76 containers-submodule, §11.4.77 re-obtain mechanism.

Evidence discipline (§11.4.6): every recommendation is cited with a URL + access date below. Claims that could not be pinned to a primary/authoritative 2025–2026 source are marked **UNCONFIRMED**. Automatic MT metrics (COMET/BLEU) are treated as *directional*, not ground truth — see the “COMET contamination” caveat in §1.

0. TL;DR — Recommended local stack

Layer	Choice	Why
Default workhorse NMT (200 langs)	NLLB-200-3.3B, served via CTranslate2 int8	Best quality-per-VRAM for broad coverage; ~3–4 GB VRAM int8; 3–4× faster than FP32. Dedicated NMT, deterministic, cheap.
Quality tier (high-resource pairs, doc-level, instruction/context-aware)	TOWER+ 9B (Unbabel, open-weight) via vLLM, or Gemma-3-27B / Qwen routed to HelixLLM	XCOMET-XXL 84.38 over 24 pairs; instruction-following + terminology adherence; ~18 GB FP16 fits the 5090.
Long-tail coverage (400+ langs)	MADLAD-400-3B/7B/10B via CTranslate2 int8	419 languages; strong on niche/legal domains; fills NLLB’s gaps.

Layer	Choice	Why
Speech / multimodal (optional)	SeamlessM4T v2 Large	Text + speech (S2T/S2S) if HelixLLM needs audio translation.
Serving front door	LibreTranslate-compatible /translate + custom /translate/ document + router to HelixLLM	Drop-in client compatibility; format-aware + glossary layer on top.
Cloud fallback / parity	Unified adapter over DeepL, Google Cloud Translation v3, Azure Translator, Amazon Translate	Same request/response shape; per-request engine selection + failover.

The RTX 5090’s 32 GB comfortably co-hosts the CTranslate2 NMT tier (single-digit GB) **and** one 9B–27B LLM tier simultaneously.

1. Local translation models — landscape (2025–2026)

1.1 Comparison table

Model	Type	Languages	Quality (headline)	VRAM (approx.)	Best serving
NLLB-200 3.3B (Meta)	Dedicated NMT (enc-dec)	200	+44% avg BLEU vs prior SoTA; low-resource +70%; fine-tuned 53.13 BLEU EN → FR medical (TICO-19) beat Llama-3.1-405B 1-shot	~8 GB FP16 / ~3–4 GB int8	CTranslate2
NLLB-200 1.3B distilled	Dedicated NMT	200	“Sweet spot” quality/resource	~2 GB int8	CTranslate2

Model	Type	Languages	Quality (headline)	VRAM (approx.)	Best serving
MADLAD-400 3B/7B/10B (Google)	Dedicated NMT (T5)	419	XCOMET legal (SwiLTra): 3B 86.82 / 7B 87.40 / 10B 86.65 — beat GPT-4 on that benchmark	3B ~7 GB FP16 (int8 ~3–4 GB); 10B fits 5090	CTranslate2 (int8; no 4-bit)
SeamlessM4T v2 Large (Meta)	Multimodal (text+speech)	~200 text / 100 speech	Matches NLLB-3.3B into-English, +1 chrF+ + from-English on FLORES; +20% BLEU speech	~6 GB	HF Transformers
TOWER+ 2B/ 9B/72B (Unbabel, open-weight)	Translation-tuned LLM	27 langs / 47 pairs	9B XCOMET-XXL 84.38 (24 pairs); 72B 83.29 WMT24++; matches/beats GPT-4o-1120, Claude-3.7	2B ~5 GB / 9B ~18 GB / 72B (multi-GPU)	vLLM
X-ALMA (ALMA family)	Translation LLM w/ plug-in modules	~50 (grouped)	ICLR 2025; “quality translation at scale” via language-group modules	13B-class ~ FP16 26 GB / int4 ~8 GB	vLLM / llama.cpp
Gemma-3-27B (Google)	General multilingual LLM	140+	WMT24++ COMET22 ~83.1 (27B) per TranslateGemma	~18 GB int4 / ~54 GB FP16	vLLM / llama.cpp

Model	Type	Languages	Quality (headline)	VRAM (approx.)	Best serving
			report (UNCONFIRMED exact number)		
Qwen 2.5/3 (7B–32B)	General multilingual LLM	29+	32B ~95% of cloud quality ZH ↔ EN; natural/idiomatic; JA/KO 80–90%	7B ~5 GB / 32B ~20 GB	vLLM / Ollama
Opus-MT (Helsinki)	Per-pair NMT	1 pair/model	Fast, lightweight	~300 MB/direction	CTranslate2

Sources: NLLB [Meta arXiv 2207.04672], MADLAD [arXiv 2309.04662], SeamlessM4T [arXiv 2308.11596], TOWER+ [Unbabel/MarkTechPost 2025-06-27], CTranslate2 [OpenNMT], InsiderLLM guide, WMT/leaderboard aggregator — all in the Sources footer.

1.2 Which give *truly professional* quality, for what

- **High-resource European pairs (EN ↔ DE/FR/ES/IT/PT/NL...):** TOWER+ 9B, DeepL-class quality; NLLB-3.3B is very strong and far cheaper. Frontier LLMs (Gemini/GPT/Claude) still edge ahead on *human* evaluation for general content, but the gap is small and closed by TOWER+ for translation-specific work.
- **CJK (ZH/JA/KO):** Qwen (32B) or TOWER+ 9B; GPT-4o-class open models beat DeepL on Asian pairs per BLEU (EN-ZH GPT-4o 57.4 vs DeepL 51.3; EN-JA 54.8 vs 49.1).
- **Low-resource / long tail (African, South/SE-Asian, indigenous):** NLLB-200-3.3B and MADLAD-400 are the professional choice — they *match or beat Google* on many such languages and cover pairs cloud APIs don't. MADLAD extends to 419 langs.
- **Legal/medical/technical with terminology control:** MADLAD-400 (legal XCOMET leader), or TOWER+/Gemma with a glossary-constrained prompt + a terminology post-check.
- **Indian languages:** consider **IndicTrans2** (22 scheduled Indian languages) as a specialist supplement [arXiv 2305.16307].

1.3 Critical caveat — don't trust COMET/BLEU blindly (§11.4.107 analogue)

The 2026 MT leaderboard aggregator explicitly warns “**COMET now has a contamination problem,**” and **Tower-v2-70B ranked #1 on COMET across all 11 WMT24 pairs but lost to Claude-3.5 on 9 of them under human evaluation** — a textbook metric-gaming case. **Operative rule for HelixLLM:** treat automatic scores as a *screen*, gate professional claims on human/MQM spot-checks or an LLM-as-judge (e.g., CometKiwi reference-free QE + a HelixLLM adequacy/fluency rubric), and never ship a “professional quality” claim on COMET alone.

2. Serving — how to expose translation behind a clean local API

2.1 CTranslate2 is the efficiency pick for NMT — CONFIRMED

CTranslate2 is a purpose-built inference engine for Transformer models (supports NLLB, MADLAD/T5, Opus-MT, M2M-100). Verified benefits: - **int8 quantization** → ~3–4× **faster inference at half the memory** vs FP32 (measured ~3.53× vs float32); **NLLB-200 handles 200 languages in ~3 GB VRAM** at int8. - Ready-made converted weights exist (e.g., OpenNMT/nllb-200-3.3B-ct2-int8). - **Limitation:** CTranslate2 supports int8/float16/int16 but **not 4-bit** — for 4-bit you'd move that model to llama.cpp/vLLM. For NMT this doesn't matter (int8 is the sweet spot).

→ **Use CTranslate2 for the dedicated NMT tier (NLLB, MADLAD, Opus-MT).** Use vLLM for the LLM tier (TOWER+/Gemma/Qwen) because it gives paged-attention throughput and OpenAI-compatible serving.

2.2 Blackwell / RTX 5090 serving gotchas (§11.4.99)

- **vLLM on RTX 5090 needs CUDA 12.8+ and a recent PyTorch (torch 2.9.0 cu128 confirmed working; torch ≥2.6/cu128 minimum).** Stock/older vLLM wheels fail because PyTorch shipped only sm_50–sm_90; sm_120 (Blackwell) requires the newer stack — build from source or use a cu128 image.
- **FlashAttention 3 does not work on Blackwell yet — set VLLM_FLASH_ATTN_VERSION=2.**
- Reported throughput: ~140 tok/s Qwen3-14B-AWQ; GPT-OSS-20B MXFP4 319–424 tok/s @8k. (Directional; **UNCONFIRMED** for our exact models.)
- **CTranslate2** must also be built/installed against a CUDA 12.8 runtime for Blackwell; verify GPU int8 kernels load (fall back to CPU int8 is possible but slow). **UNCONFIRMED** whether prebuilt CT2 wheels ship sm_120 kernels as of 2026-07 — pin the build in the container and test nvidia-smi-visible GPU inference at image-build time.

2.3 API shape (LibreTranslate-compatible + extensions)

LibreTranslate is the reference self-hosted OSS API (Argos Translate backend):
POST /translate with {q, source, target, format}, plus /detect, /languages, HTML translation, batch, API keys, Prometheus metrics, GPU acceleration. Adopt its request/response contract so any LibreTranslate client works unchanged, then extend:

POST /translate	# LibreTranslate-compatible (q, source
POST /translate/batch	# array in, array out
POST /translate/document	# markup/format-aware (md/html/xml/doc
POST /translate/context	# route to HelixLLM for context/docume
GET /languages	# union of all engines' supported pair
GET /engines	# nllb madlad tower gemma deep
POST /glossaries	# CRUD terminology
GET /detect	
GET /healthz /metrics	# Prometheus

Routing policy (server-side): 1. engine explicit → use it. 2. else pick by (source, target) + domain + latency SLA: dedicated NMT (CT2) for broad/low-resource + throughput; TOWER+/HelixLLM for high-resource + doc-level + terminology; cloud only on explicit request or local-failure failover. 3. Optional **QE gate**: run CometKiwi/HelixLLM judge; if score < threshold, escalate to the quality tier or a cloud engine, log the escalation as captured evidence.

3. Document/format-aware translation + glossary/terminology

Never translate raw markup with an LLM and hope. Use tokenize-translate-restore:

- **Markdown/code/LaTeX/links**: protect code blocks, inline code, LaTeX, links, image paths, HTML/JSX tags by replacing them with unique placeholder tokens, translate the prose only, then restore. OSS reference: **md-translator** (tokenizes those constructs, 25+ engines). For PDFs, **BabelDOC** does layout-preserving translation via an intermediate representation with terminology-constraint + math placeholders.
- **HTML/XML**: use real tag handling — DeepL exposes tag_handling=html|xml, ignore_tags, outline_detection; emulate the same on the local side (parse → protect non-translatable subtrees → translate text nodes → reassemble). For placeholder tags (Mustache/ICU {name}), pre/post-process to XML placeholders with unique IDs so the engine preserves them.

- **DOCX/PPTX/XLSX**: extract runs, translate text, re-inject (or front a library); preserve inline formatting runs as placeholders.

Glossary/terminology control: - **Dedicated NMT (NLLB/MADLAD via CT2)**: constrained decoding / do-not-translate lists + a deterministic terminology post-substitution pass (glossary as source → target token map, applied after protecting placeholders). - **LLM tier (TOWER+/Gemma/HelixLLM)**: inject the glossary into the instruction (“Use these translations for the following terms: ...”) — TOWER+ is explicitly instruction-following, which is its differentiator. Follow with a terminology-adherence check (regex/fuzzy match on required target terms); on miss, re-prompt or fall back. - Store glossaries per (client, domain, langpair); expose CRUD via /glossaries. Map to each cloud engine’s native glossary at adapter level (§4).

4. Cloud provider coverage — unified adapter

All four are `text[] → text[]` with per-engine auth, glossary, and formality knobs. Normalize to the internal /translate contract; keep secrets in .env (§CONST-042, mode 0600), never hardcoded.

Provider	Endpoint / auth	Glossary	Formality / extras
DeepL	POST <code>https://api.deepl.com/v2/translate</code> ; header <code>Authorization: DeepL-Auth-Key <key></code> ; body <code>text[], target_lang, glossary_id, tag_handling</code>	glossary_id (declension/gender/tense-aware — best-in-class)	formality, ignore_tags, HTML/XML handling
Google Cloud Translation v3	<code>projects.locations.translateText</code> (REST/gRPC), OAuth/service-account	glossaryConfig (CSV/TSV term pairs)	model selection (NMT/LLM), 130+ langs
Azure Translator	POST <code>{endpoint}/translate?api-version=3.0</code> (GA) or new preview 2025-05-01; header <code>Ocp-Apim-Subscription-Key (+region)</code>	Custom Translator terminology models	profanity, transliteration, dictionary lookup
Amazon Translate	AWS SDK <code>TranslateText / TranslateDocument</code> , SigV4	custom terminology	formality, profanity masking, 75 langs

Adapter design: one interface `Translate(ctx, texts, src, tgt, opts) → ([]string, meta)`; per-provider structs implement it; a factory selects by engine. Map internal glossary → provider-native glossary at upload time (cache glossary IDs). Implement retry/backoff, rate-limit handling, and per-request cost accounting. Prefer batch endpoints. (Verify each provider's *latest* request shape at build time per §11.4.99 — DeepL and Azure both revised APIs in 2025.)

5. Rootless Podman deployment (§11.4.161, §11.4.76, §11.4.77)

5.1 GPU access under rootless Podman — CDI, not `--gpus`

Use the **NVIDIA Container Device Interface (CDI)**. As of NVIDIA Container Toolkit **v1.18.0**, `nvidia-cdi-refresh` (systemd) auto-generates `/var/run/cdi/nvidia.yaml`. For rootless, generate in user space:

```
# one-time host prep (toolkit installed)
mkdir -p ~/.config/cdi
nvidia-ctk cdi generate --output=$HOME/.config/cdi/nvidia.yaml
# run (rootless)
podman --cdi-spec-dir=$HOME/.config/cdi run --rm \
  --device nvidia.com/gpu=all --security-opt=label=disable \
  <image> nvidia-smi -L
```

Known rootless friction: CDI historically worked in privileged but not rootless on some setups (podman#17539); on modern toolkit (≥ 1.16 , ideally ≥ 1.18) + Podman ≥ 4.1 (CDI-in-`--device`) it works. Verify at image-build/boot with `nvidia-smi -L` (positive evidence, §11.4.5).

5.2 Images & volumes

- **Base image:** CUDA 12.8 runtime (Blackwell) — e.g., an `nvidia/cuda:12.8.x-runtime` derivative; install CTranslate2 (CUDA build) + vLLM (torch 2.9 cu128) + FastAPI service. Two images recommended: `helix-mt-nmt` (CTranslate2) and `helix-mt-llm` (vLLM), composed via the **containers submodule** `pkg/boot/pkg/compose/pkg/health` (§11.4.76) — no ad-hoc `podman run` in workflows.
- **Model volumes:** mount a host model dir (`-v $HOME/helix/models:/models:Z`); models are large build derivatives → **git-ignored** (§CONST-053) with a §11.4.77 **re-obtain mechanism**: a `scripts/fetch_models.sh` that pulls from Hugging Face (nllb-200-3.3B-ct2-int8, MADLAD ct2, TOWER+9B) with pinned revisions + SHA256, plus a `.gitignore-meta/<slug>.yaml` declaring source URL, disk budget, and integrity hash. First-

run bootstrap in `scripts/setup.sh`; a pre-build gate checks presence or a `.regenerated/<slug>.ok` stamp.

- **Rootless niceties:** run as non-root UID in-container; use `:Z` SELinux relabel on mounts; keep `--security-opt=label=disable` only if SELinux blocks the CDI device (prefer proper labeling).
-

6. Recommended concrete build

1. `helix-mt-nmt` container: FastAPI + CTranslate2, serving **NLLB-200-3.3B int8** (default) + **MADLAD-400 int8** (long tail) + Opus-MT micro-pairs. LibreTranslate-compatible `/translate`.
 2. `helix-mt-llm` container: vLLM (cu128, FA v2) serving **TOWER+ 9B** for high-resource/doc/terminology; exposes OpenAI-compatible + is also reachable by the router as an “engine.”
 3. `helix-mt-gateway`: the router + `/translate/document` (tokenize-restore) + `/glossaries` + QE gate + **cloud adapters** (DeepL/Google/Azure/AWS) behind the same contract.
 4. HelixLLM integration: `/translate/context` hands document + glossary + surrounding context to HelixLLM for context-aware translation and code-comment/string-literal translation in dev workflows.
 5. Compose via containers-submodule; models fetched by §11.4.77 script; GPU via rootless CDI.
-

7. Top risks

1. **Blackwell toolchain drift (highest).** RTX 5090 sm_120 requires CUDA 12.8 + torch 2.9/cu128; FA3 unsupported; CTranslate2 GPU int8 kernels for sm_120 are **UNCONFIRMED** as prebuilt. Mitigation: pin a working image, prove GPU inference at build time, keep a CPU-int8 fallback for CT2 and a cloud-engine failover.
 2. **Metric-gaming / false “professional quality” (§11.4.107).** COMET/BLEU can be green while human quality is not (Tower-v2-70B case). Mitigation: mandatory QE + human/MQM spot-check gate before any professional claim; per-langpair quality ledger.
 3. **Glossary/markup breakage in document translation.** LLMs drop/alter placeholders and tags; terminology not honored. Mitigation: deterministic tokenize-protect-restore, terminology post-check with re-prompt/fallback, golden-file regression tests on md/html/xml/docx (paired mutation per §1.1).
-

Sources verified 2026-07-06

- Meta, *No Language Left Behind* (NLLB-200) — <https://arxiv.org/pdf/2207.04672> (accessed 2026-07-06)
- Google, *MADLAD-400* — <https://arxiv.org/pdf/2309.04662> (accessed 2026-07-06)
- Meta, *SeamlessM4T* — <https://arxiv.org/pdf/2308.11596> (accessed 2026-07-06)
- IndicTrans2 — <https://arxiv.org/pdf/2305.16307> (accessed 2026-07-06)
- Unbabel TOWER+ overview (MarkTechPost, 2025-06-27) — <https://www.marktechpost.com/2025/06/27/unbabel-introduces-tower-a-unified-framework-for-high-fidelity-translation-and-instruction-following-in-multilingual-llms/> (accessed 2026-07-06)
- Unbabel, *Announcing Tower* — <https://unbabel.com/announcing-tower-an-open-multilingual-llm-for-translation-related-tasks/> (accessed 2026-07-06)
- X-ALMA (ICLR 2025) — referenced via Unbabel/OpenReview; see leaderboard aggregator (accessed 2026-07-06)
- Machine Translation Benchmarks Leaderboard 2026 (aggregator; COMET-contamination + WMT25 human-eval note) — <https://awesomeagents.ai/leaderboards/translation-benchmarks-leaderboard/> (accessed 2026-07-06)
- InsiderLLM, *Best Local LLMs for Translation* (VRAM/engine table) — <https://insiderllm.com/guides/best-local-llms-translation/> (accessed 2026-07-06)
- SwiLTra-Bench (legal MT, MADLAD XCOMET) — <https://arxiv.org/pdf/2503.01372> (accessed 2026-07-06)
- CTranslate2 (OpenNMT) repo — <https://github.com/OpenNMT/CTranslate2> (accessed 2026-07-06)
- CTranslate2 quantization docs — <https://opennmt.net/CTranslate2/quantization.html> (accessed 2026-07-06)
- OpenNMT nllb-200-3.3B-ct2-int8 — <https://huggingface.co/OpenNMT/nllb-200-3.3B-ct2-int8> (accessed 2026-07-06)
- LibreTranslate repo/docs — <https://github.com/LibreTranslate/LibreTranslate> , <https://docs.libretranslate.com/> (accessed 2026-07-06)
- md-translator (markup-preserving) — <https://github.com/rockbenben/md-translator> (accessed 2026-07-06)
- BabelDOC (layout-preserving PDF translation) — <https://arxiv.org/html/2605.10845v1> (accessed 2026-07-06; **note: future-dated arXiv id, treat as UNCONFIRMED primary**)
- DeepL API — translate reference <https://developers.deepl.com/api-reference/translate> ; XML/HTML handling <https://developers.deepl.com/docs/xml-and-html-handling/xml> ; placeholder tags <https://developers.deepl.com/docs/resources/examples-and-guides/placeholder-tags> (accessed 2026-07-06)
- Google Cloud Translation v3 / Azure Translator / Amazon Translate comparison — <https://chatscontrol.com/blog/deepl-api-vs-google-cloud-vs-azure-translator-comparison> , <https://kitemetric.com/blogs/pick-the-perfect-translation-api-in-2025-a-guide-for-developers> (accessed 2026-07-06)

- NVIDIA Container Toolkit CDI support — <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/cdi-support.html> (accessed 2026-07-06)
- Run NVIDIA GPU containers with Podman (rootless CDI, nvidia-cdi-refresh) — <https://oneuptime.com/blog/post/2026-03-18-run-nvidia-gpu-containers-podman/view> ; Podman Desktop GPU — <https://podman-desktop.io/docs/podman/gpu> (accessed 2026-07-06)
- Podman rootless CDI issue (#17539) — <https://github.com/containers/podman/issues/17539> (accessed 2026-07-06)
- vLLM RTX 5090 / Blackwell setup — <https://github.com/vllm-project/vllm/issues/13306> , <https://discuss.vllm.ai/t/vllm-on-rtx5090-working-gpu-setup-with-torch-2-9-0-cu128/1492> , <https://github.com/vllm-project/vllm/issues/14452> (accessed 2026-07-06)

Negative findings / gaps (§11.4.99(B)): (1) No authoritative confirmation that prebuilt CTranslate2 wheels ship sm_120 (Blackwell) GPU kernels as of 2026-07 — must be verified at container build. (2) Several 2026 arXiv ids surfaced by search aggregators (TranslateGemma 2601.09012, Tower scaling 2602.11961, BabelDOC 2605.10845) are future-dated and could not be cross-verified against a stable primary; exact COMET numbers from them are marked UNCONFIRMED. (3) TOWER+ per-language pair list beyond “27 langs / 47 pairs” not enumerated here — check the HF model card before committing to a language matrix.